

Mise en place de minikube (impossible de faire du scaling sans ça visiblement)

chercher le .exe sur : [minikube start | minikube](#)

et suivre la doc qui suis

A faire une fois docker desktop lancé , ou celui que vous avez, mais il doit être lancé une fois installé : minikube start

```
kubectl apply -f db-us-deployment.yaml
```

```
kubectl apply -f db-us-service.yaml
```

```
kubectl apply -f db-ch-deployment.yaml
```

```
kubectl apply -f db-ch-service.yaml
```

```
kubectl apply -f database-deployment.yaml
```

```
kubectl apply -f database-service.yaml
```

```
kubectl apply -f backend-fr-deployment.yaml
```

```
kubectl apply -f backend-fr-service.yaml
```

```
kubectl apply -f backend-us-deployment.yaml
```

```
kubectl apply -f backend-us-service.yaml
```

```
kubectl apply -f backend-ch-fr-deployment.yaml
```

```
kubectl apply -f backend-ch-fr-service.yaml
```

```
kubectl apply -f backend-ch-en-deployment.yaml
```

```
kubectl apply -f backend-ch-en-service.yaml
```

```
kubectl apply -f backend-ch-de-deployment.yaml
```

```
kubectl apply -f backend-ch-de-service.yaml
```

```
kubectl apply -f frontend-deployment.yaml
```

```
kubectl apply -f frontend-service.yaml
```

pour tout lancer d'un coup : `kubectl apply -f c:\Users\gaels\OneDrive\Documents\ECOLE-
EPSI\Mspr\k8s_manifests`

ou sans le chemin si tu es déjà dedans

mise en place dans les dockerfile :

commande pour builder et charger dans minikube

Backend (exemple)

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\backend
```

```
docker build -t my_backend_image:latest -f dockerfile .
```

```
minikube image load my_backend_image:latest
```

Frontend

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\frontend
```

```
docker build -t my_frontend_image:latest -f dockerfile .
```

```
minikube image load my_frontend_image:latest
```

Reverse proxy

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\essaidocker
```

```
docker build -t my_reverse_proxy:latest .
```

```
minikube image load my_reverse_proxy:latest
```

« accède à ton appli via le service du reverse proxy

```
minikube service <nom-du-service-reverse-proxy>
```

Active l’Ingress Controller dans Minikube :

```
minikube addons enable ingress
```

Déploie l’Ingress :

```
kubectl apply -f ingress.yaml
```

déploiement des bases:

```
kubectl apply -f db-us-deployment.yaml
```

```
kubectl apply -f db-us-service.yaml
```

```
kubectl apply -f db-ch-deployment.yaml
```

```
kubectl apply -f db-ch-service.yaml
```

```
kubectl apply -f database-deployment.yaml
```

```
kubectl apply -f database-service.yaml
```

#déploie tous les fichiers YAML kubernetes – peut remplacer toutes les commandes de déploiement précédentes

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\k8s_manifests
```

```
kubectl apply -f .
```

OU

```
kubectl apply -f c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\k8s_manifests
```

A partir de la, fini le docker compose, on s'en cogue !!!

Maintenant que tout est déployé on vérifie que tout est bien nickel (déployé et en fonctionnement) avec qq commandes de contrôle :

```
kubectl get pods
```

```
kubectl get svc
```

```
kubectl get ingress
```

cette commande ne marchera que si on a bien activé l'ingress

on doit ensuite récupérer l'ip de minikube c'est ce qui va nous permettre d'accéder à l'application

```
minikube ip
```

on accède à notre application dans le navigateur :

- Pour le frontend : `http://<IP-minikube>/`
- Pour les API : `http://<IP-minikube>/api/fr/...`, `/api/us/...`, etc.

Eventuellement, c'est bon de les connaître : pour arrêter ou nettoyer avant un nouveau build :

```
kubectl delete -f .
```

OU

minikube stop

a partir de la j'ai eu quelques soucis avec le conteneur minikube qui ne restaient pas allumés.
J'ai désinstallé minikube avec

```
kubectrl delete all --all  
minikube stop
```

minikube delete

et on recommence :

```
minikube start --driver=docker
```

```
minikube addons enable ingress
```

on relance les trois éléments (normalement si vous reprenez bien ce que j'ai fais, vous ne devriez qu'avoir vos chemin à adapter) *et on ne se précipite pas entre les commandes !!! je me suis arraché les cheveux parce que les builds merdaient vu que j'enchainait trop vite les commandes, technologie de CHIOTTES :

Backend (exemple)

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\backend
```

```
docker build -t my_backend_image:latest .
```

```
minikube image load my_backend_image:latest
```

```
kubectrl rollout restart deployment backend-fr
```

```
kubectrl rollout restart deployment backend-us
```

```
kubectrl rollout restart deployment backend-ch-fr
```

```
kubectrl rollout restart deployment backend-ch-en
```

```
kubectrl rollout restart deployment backend-ch-de
```

Frontend

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\frontend
```

```
docker build -t my_frontend_image:latest .
```

```
minikube image load my_frontend_image:latest
```

```
kubectrl rollout restart deployment frontend
```

Reverse proxy

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\essaidocker
```

```
docker build -t my_reverse_proxy:latest -f Dockerfile .
```

```
minikube image load my_reverse_proxy:latest
```

```
kubectl rollout restart deployment reverse-proxy
```

Quand c'est fait on retourne dans le dossier de manifest (la commande suivante est l'équivalente des rollout précédents) :

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\k8s_manifests
```

```
kubectl apply -f .
```

on verifie que tout va bien et qu'ils sont bien lancés

```
kubectl get pods
```

ET CA N'A JAMAIS FOCNTIONNE !

Je tente avec Kind du coup

J'ai aussi tenté en changeant le nom des images (ligne : image :my_frontend_image dans le fichier deployment de k8s, et je l'ai changé en utilisant des images publique, genre alpine)

Arrêter Minikube

```
minikube stop
```

Supprimer complètement Minikube

```
minikube delete
```

Installation avec Chocolatey (si pas déjà installé) il faut passer par un terminal admin

```
choco install kind
```

Créer un cluster simple

```
kind create cluster --name mspr
```

Vérifier que le cluster est bien créé

```
kubectl get nodes
```

Vérifier le contexte (doit être kind-mspr)

kubectrl config current-context

Précharger les images publiques (optionnel)

kind load docker-image nginx:alpine --name mspr

kind load docker-image tiangolo/uvicorn-gunicorn-fastapi:python3.9 --name mspr

kind load docker-image postgres:latest --name mspr

----- attention point sensible ---- j'ai eu des soucis car mauvaise co (merci le train) et si les commandes suivantes ne marchent pas vous pouvez faire la methode 2 qui vient après

Build des images (si pas déjà fait)

cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\backend

docker build -t my_backend_image:latest .

cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\frontend

docker build -t my_frontend_image:latest .

Chargement dans Kind

kind load docker-image my_backend_image:latest --name mspr

kind load docker-image my_frontend_image:latest --name mspr

kind load docker-image nginx:alpine --name mspr # Pour le reverse proxy

cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\k8s_manifests

Déployer les ConfigMaps d'abord

kubectrl apply -f nginx-config.yaml

Puis les bases de données

kubectrl apply -f database-deployment.yaml -f database-service.yaml -f db-ch-deployment.yaml
-f db-ch-service.yaml -f db-us-deployment.yaml -f db-us-service.yaml

Puis les backends et services

```
kubectl apply -f backend-deployment.yaml -f backend-service.yaml -f backend-us-  
deployment.yaml -f backend-us-service.yaml -f backend-ch-fr-deployment.yaml -f backend-ch-  
fr-service.yaml -f backend-ch-en-deployment.yaml -f backend-ch-en-service.yaml -f backend-  
ch-de-deployment.yaml -f backend-ch-de-service.yaml
```

Et enfin le frontend et le reverse proxy

```
kubectl apply -f frontend-deployment.yaml -f frontend-service.yaml -f reverse-proxy-  
deployment.yaml
```

Créer un port-forward vers le service reverse-proxy

```
kubectl port-forward svc/reverse-proxy-service 8080:80
```

----- 2 eme methode -----

Va dans le dossier de tes manifests

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\k8s_manifests
```

Déploie directement tous les fichiers YAML

```
kubectl apply -f .
```

Vérifie l'état des pods (ils vont prendre un peu de temps à démarrer au premier lancement)

```
kubectl get pods
```

es pods vont passer de Pending → ContainerCreating → Running. Ce processus peut prendre 1-2 minutes lors du premier déploiement car Kind télécharge les images. Il est possible de devoir faire plusieurs fois la commande

Surveille les pods en temps réel (seulement si ça vous amuse)

```
kubectl get pods -w
```

Port-forward pour accéder à ton application

```
kubectl port-forward svc/reverse-proxy-service 8080:80
```

Puis ouvre <http://localhost:8080> dans ton navigateur.

Je tombe sur la page d'accueil de nginx... ce truc aura ma peau

donc faut créer un nouveau fichier de config nginx : il s'appelle nginx-config-fixed.yaml

```
kubectl apply -f nginx-config-fixed.yaml
```

```
kubectrl rollout restart deployment reverse-proxy
```

puis après quelques seconds :

```
kubectrl port-forward svc/reverse-proxy-service 8080:80
```

-----V3

```
# FRONTEND
```

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\frontend
```

```
docker build -t my_frontend_image:latest .
```

```
kind load docker-image my_frontend_image:latest --name mspr
```

```
# BACKEND
```

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\backend
```

```
docker build -t my_backend_image:latest .
```

```
kind load docker-image my_backend_image:latest --name mspr
```

```
#REVERSE PROXY
```

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\k8s_manifests
```

```
kubectrl apply -f reverse-proxy-service.yaml
```

```
cd c:\Users\gaels\OneDrive\Documents\ECOLE-EPSI\Mspr\k8s_manifests
```

```
kubectrl apply -f .
```

```
kubectrl rollout restart deployment frontend
```

```
kubectrl rollout restart deployment reverse-proxy
```

```
kubectrl rollout restart deployment backend-fr backend-us backend-ch-fr backend-ch-en  
backend-ch-de
```

```
kubectrl get pods
```

```
kubectrl port-forward svc/frontend-service 8081:80
```

```
kubectrl port-forward svc/reverse-proxy-service 8080:80
```


#init la bdd

```
kubectl create configmap initdb-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\initdb\dump.sql
```

```
kubectl create configmap initch-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\init_ch\dumpch.sql
```

```
kubectl create configmap initus-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\init_us\dumpus.sql
```

-----kubectl delete deployment --all

kubectl delete svc --all

kubectl delete configmap --all

kubectl delete pod --all-----

```
kubectl create configmap initdb-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\initdb\dump.sql
```

```
kubectl create configmap initch-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\init_ch\dumpch.sql
```

```
kubectl create configmap initus-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\init_us\dumpus.sql
```

----RESUM2----

1. Rebuild et recharge les images custom

cd ../frontend

docker build -t my_frontend_image:latest .

kind load docker-image my_frontend_image:latest --name mspr

cd ../backend

```
docker build -t my_backend_image:latest .
```

```
kind load docker-image my_backend_image:latest --name mspr
```

2. Redémarre les déploiements

```
kubectl rollout restart deployment frontend
```

```
kubectl rollout restart deployment backend-fr backend-us backend-ch-fr backend-ch-en  
backend-ch-de
```

3. Réapplique le service reverse proxy si besoin

```
kubectl apply -f reverse-proxy-service.yaml
```

4. Vérifie les pods

```
kubectl get pods
```

5. Port-forward pour tester

```
kubectl port-forward svc/reverse-proxy-service 8080:80
```

```
kubectl create configmap init-us-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\init_us\dumpus.sql  
kubectl create configmap init-ch-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\init_ch\dumpch.sql  
kubectl create configmap initdb-sql --from-  
file=init.sql=c:\Users\gaels\OneDrive\Documents\ECOLE-  
EPSI\Mspr\essaidocker\initdb\dump.sql
```

Documentation de déploiement Kubernetes

Prérequis

- **Docker Desktop** installé et démarré (avec support Kubernetes ou WSL2 activé)
- **kubectl** installé et accessible dans le PATH
- **Kind** installé (go install sigs.k8s.io/kind@latest ou via [la doc officielle](#))
- **Git Bash** ou **WSL** pour exécuter des scripts Bash si besoin

1. Cloner le projet

```
git clone <url-du-repo>
```

```
cd <nom-du-repo>
```

2. Créer un cluster Kubernetes local avec Kind

```
kind create cluster --name mspr-cluster
```

3. Construire les images Docker

Dans le dossier du projet :

```
docker build -t my_backend_image:latest ./backend
```

```
docker build -t my_frontend_image:latest ./frontend
```

4. Charger les images dans Kind

```
kind load docker-image my_backend_image:latest --name mspr-cluster
```

```
kind load docker-image my_frontend_image:latest --name mspr-cluster
```

5. Créer les ConfigMap pour l'initialisation des bases

```
kubectl create configmap initdb-sql --from-file=essaيدocker/initdb/dump.sql --dry-run=client -  
o yaml | kubectl apply -f -
```

```
kubectl create configmap init-us-sql --from-file=essaيدocker/init_us/dumpus.sql --dry-  
run=client -o yaml | kubectl apply -f -
```

```
kubectl create configmap init-ch-sql --from-file=essaيدocker/init_ch/dumpch.sql --dry-  
run=client -o yaml | kubectl apply -f -
```

6. Déployer tous les manifests Kubernetes

```
kubectl apply -f k8s_manifests/
```

7. Vérifier le statut des pods

```
kubectl get pods
```

Attendre que tous les pods soient en Running.

8. Accéder à l'application

Avec NodePort

- Récupérer le port NodePort du frontend ou du reverse-proxy :

kubectl get svc

- Accéder à :
http://localhost:<nodeport>
(ex : <http://localhost:31362>)

Avec port-forward (recommandé sur Windows/WSL2)

kubectl port-forward svc/reverse-proxy-service 8080:80

Puis ouvrir : <http://localhost:8080>

9. Sauvegarde et restauration des bases

- Utiliser le menu du script [init_and_deploy.bat](#) pour lancer une sauvegarde ou une restauration.
- Les dumps sont stockés dans le dossier [sauvegardes_bdd](#).

10. Nettoyer le cluster (optionnel)

Pour supprimer le cluster Kind :

kind delete cluster --name mspr-cluster

Conseils

- Pour chaque modification du code backend/frontend, rebuild l'image, recharge-la dans Kind, puis redéploie les pods concernés.
- Utilise les scripts fournis pour automatiser les sauvegardes/restaurations.
- Consulte les logs avec :

kubectl logs <nom-du-pod>